

```

1  module ShiftRightRegister (
2      input wire      clk,
3      input wire      reset,
4      input wire      load,
5      input wire [23:0] adder_outL,
6      input wire [23:0] adder_outR,
7      output reg [39:0] shift_outL,
8      output reg [39:0] shift_outR,
9      output reg      done_shifting
10 );
11
12     reg [5:0] shift_count;
13     reg      shifting;
14
15     always @(posedge clk or posedge reset) begin
16         if (reset) begin
17             shift_outL    <= 40'd0;
18             shift_outR    <= 40'd0;
19             shift_count    <= 6'd0;
20             shifting      <= 1'b0;
21             done_shifting <= 1'b0;
22         end else begin
23             done_shifting <= 1'b0;
24
25             if (load) begin
26                 shift_outL[39:16] <= adder_outL;
27                 shift_outL[15:0]  <= 16'd0;
28                 shift_outR[39:16] <= adder_outR;
29                 shift_outR[15:0]  <= 16'd0;
30                 shift_count      <= 6'd0;
31                 shifting         <= 1'b1;
32             end else if (shifting) begin
33                 shift_outL <= {1'b0, shift_outL[39:1]};
34                 shift_outR <= {1'b0, shift_outR[39:1]};
35                 shift_count <= shift_count + 1;
36
37                 if (shift_count == 6'd31) begin
38                     shifting      <= 1'b0;
39                     done_shifting <= 1'b1;
40                 end
41             end
42         end
43     end
44
45 endmodule
46
47
48
49 //Testbench for the ShiftRightRegister
50 `timescale 1ns / 1ps
51
52 module ShiftRightRegister_tb;
53
54     // Inputs
55     reg clk, reset, load;
56     reg [23:0] adder_outL, adder_outR;
57
58     // Outputs
59     wire [39:0] shift_outL, shift_outR;
60     wire done_shifting;
61
62     // Instantiate DUT
63     ShiftRightRegister UUT (
64         .clk(clk),
65         .reset(reset),
66         .load(load),
67         .adder_outL(adder_outL),
68         .adder_outR(adder_outR),
69         .shift_outL(shift_outL),

```

```

70     .shift_outR(shift_outR),
71     .done_shifting(done_shifting)
72 );
73
74 // Clock generator
75 always #5 clk = ~clk; // 10 ns clock period
76
77 initial begin
78     // Initialize signals
79     clk = 0;
80     reset = 1;
81     load = 0;
82     adder_outL = 24'hABCDE0;
83     adder_outR = 24'h123450;
84
85     // Reset pulse
86     #10 reset = 0;
87
88     // Load input values
89     #10 load = 1;
90     #10 load = 0; // Clear load signal
91
92     // Wait for done_shifting
93     wait (done_shifting);
94     $display("Shifting done at time %0t", $time);
95     $display("Final shift_outL = 0x%h", shift_outL);
96     $display("Final shift_outR = 0x%h", shift_outR);
97
98     #20 $finish;
99 end
100
101 // Monitor progress
102 initial begin
103     $monitor("T=%0t | load=%b | done=%b | shift_outL=0x%h | shift_outR=0x%h",
104             $time, load, done_shifting, shift_outL, shift_outR);
105 end
106
107 endmodule
108

```